Database Recovery

Database Recovery Management

- Database recovery restores a database from a previous consistent state
  - Necessary in the event of human error, natural disasters or hardware/software failure
- Important Concepts
  - The write-ahead-log (WAL) protocol ensures that transactions are written to the log before the database is updated
  - Redundant transaction logs ensure that multiple copies of the transaction log are stored and can be swapped
  - A database buffer is temporary in-memory storage of data being used in a transaction that is cleared once the transaction is committed
  - Checkpoints are operations in which the DBMS writes all data in buffers to memory, it happens regularly behind the scenes

Database Recovery Continued

- In a deferred-write recovery procedure, transaction operations do not immediately update the database, instead the transaction log is updated and then those events propogated to the database.
- If a failure occurs the recovery process follows these steps:

1. Identify the last checkpoint
2. Any transactions committed before the last checkpoint are safe
3. All commits made after the last checkpoint need to made in order from oldest to newest
4. Any transactions that were not committed or rolled back can be skipped because no updates were made

Database Recovery Continued

- In a deferred write-through, transactions immediately update the database even before the commit is made. A rollback is done if an transcation is subsequently aborted.
- If a failure occurs the recovery process follows these steps:

1. Identify the last checkpoint
2. Any transactions committed before the last checkpoint are safe
3. All commits made after the last checkpoint need to made in order (from oldest to newest)
4. Any transactions that were not committed or rolled back are executed in order from newest to oldest

Database and Query Optimization

Database Performance Tuning

- **Database performance tuning** is the process designed to reduce the response time of the database.
    - We give the database an input (in the form of a query) and we want it to return an output (our results set or a message) as soon as possible
    - Good database design through adequate data modelling and normalization gets us a good way to optimized performance
- The availability of RAM, CPU power and I/O (network/hard drive) throughput to the DBMS, generally have a significant influence on performance

Performance Tuning

- **SQL performance tuning** occurs on the client machine where the end user is focused on generating the correct answers to their questions in the least amount time.
    - Client side!
- **DBMS performance tuning** is focused on ensuring that the DBMS is properly configured to return results to the end user(s) as quickly and efficiently as possible.
    - Server side!

DBMS Architecture

- Recall that databases abstract away the data storage, but still use data files
- The size or **extent** of the data file is configured by the database administrator or developer and controls the regular, automated expansion of the data files.
- A **table space** is a logical storage space (on disk) used by the DBMS to group related data
    - Example: metadata, user-defined tables, indexes, etc.
- The **data cache** is the shared, reserved area of memory that stores the most recently accessed data blocks
    - The cache allows faster retrieval of the same or similar data

DBMS Architecture Continued

- The **SQL cache** is the part of memory reserved for the most recently executed SQL statements or procedural SQL (functions, stored procedures and triggers)
- Data is read or written to the database using low level data access via **input/output request**
    - The more I/O operations there are, the longer the request will take
    - Caching data helps to mitigate disk I/O requests which take the longest

Database Query Optimization Timing

- Query optimization relies on algoriths that focus on minimizing the network cost and selecting the optimum execution order
    - Automatic optimization is implemented on the DBMS and doesn't require end-user input
    - Manual optimization is selected and scheduled by the end-user
- **Static query optimization** takes place a compilation and the access strategy has already been pre-determined
- **Dynamic query optimization** creates the access strategy at runtime using the most up-to-date information

Database Query Optimization Method

- **Statisical query optimization** creates an access strategy based on the statistical information about a database
    - Example: the number of tables, rows, users, etc.
    - Statistics can be configured to generated/refresh dynamically or manually
- **Rule based query optimization** uses a set of general, preset user-defined rules to determine the best approach for executing a query

Database Statistics

- Measurements about database objects and physical hardware characteristics (CPU, memory and disk)
- Table-level statistics may include the number of rows, disk-blocks, number of columns, distinct values and more.
- Index-level statistcs include the number of distinct values in an index, a statistical distribution (hsitogram) of values and the number disk pages used
- Environment Resource statistics focus on logicial and physical disk block size, the data files size and extents

Query Processing

When a DBMS recieves a SQL statement from the client (end-user), it is processed in 3 phases

1. Parsing
2. Execution
3. Fetching

Query Parsing Phase

- The query is broken down into smaller block to enable the most efficient execution
    - Despite breaking down the original SQL statement, the same results will be returned
- The **query optimizer** in the DBMS validates syntax, validates the objects and columns, checks access rights and decomposes the SQL statement
- A SQL statement is the input to the query optimizer while the **execution plan** is the output

Query Operations and Speed

- A full table scan (reading an entire table row by row) is the slowest operation
- Sorting, `ORDER BY`, is a slow operation, made slower by a lack of an index

Query Execution Phase

- After the execution plan is created, it's passed off to actually execute
- During this phase the query is acquires the appropriate locks and the data is retrieved from the data files and placed in the cache
- Transaction managment commands (`START TRANSACTION`) are completed during this phase

Query Fetching Phase

- Rows requested in the query are returned to the client
- In this phase rows are filtered, aggregates calculated and results are ordered based on the original input

Fetching/Processing Bottlenecks

- Hardware
  - CPU, RAM or hard drive limitations may inhibit performance if they do not meet the needs of the DBMS
  - Limitations in network bandwidth causes slow the communication between the client and server
- Software
  - Poorly designed databases
  - Poor or inefficient code

Indexing and Query Optimization

Types of Indexes

- B-Tree
- Bitmat
- Hash

B-tree Index

- An index organized as upside down balanced tree where the leaves contain the pointers to the actual data
- The index is stored separately from the data
- Predominate index used in databases
- Best used when values repeat a minimal number of times (more unique) and comparisons (>, <, >=, <=)

Hash Index

- Hash values are generated by using an algorithm to assign a value to a hash table that is assigned specific location on a disk page (bucket)
- Think about retrieving a value in an array by an index
- Excels at equality based retrieval

Bitmap Index

- Uses 0s and 1s to represent the existence of values or conditions
- Best used for tables with large numbers of rows and less unique values
- Used more frequently for analytical databases

SQL Perfomance Tuning

- Index Selectivity
  - Best used when data is being filtered with `WHERE` and `HAVING` or when aggregation or sorting is done with `GROUP BY` or `ORDER BY`
  - During data modelling we start to develop an understanding of the end user needs, this can help to inform our indexing
- Conditional Expressions
  - Numeric comparisons are faster than others
  - Equality comparisons are faster than inequality comparisons
  - Avoid functions in comparison operations when possible
  - Think through your order of operations when using `AND` (FALSE first) and `OR` (TRUE first)

SQL Performance Tuning: Query Formulation

- Know your data!
- The business knows what results they want, while the technologists will know the most effecient path to getting thos results

Homework

- Read Chapter 12